MOBILE APPLICATION TECHNOLOGY / FINAL CASE STUDY

Gas Tracker

A mobile fuel log for clearer spending and station comparisons

GROUP MEMBERS	COURSE DETAILS
[Member 1 Name] [Member 2 Name] [Member 3 Name] [Member 4 Name]	Section: [Section] Instructor: Sir Gel Report date: November 7, 2026

SOURCE REPOSITORY

https://github.com/cedricangulo/gas-tracker-mobile

Course deliverable documenting the problem, application, implementation, evaluation, and improvement opportunities.

CASE STUDY

1. Problem and proposed solution

Motorcycle owners can find it difficult to keep consistent fuel records and use them to understand spending, fuel economy, and station prices. Notes kept in different places do not easily show how one fill-up relates to the next or where a useful station is located.

Gas Tracker provides one place to record fill-ups and review the history. Its main intended user is a motorcycle owner who wants to track fuel purchases and compare saved station prices. The class instance uses one shared account and one shared dataset, so all classmates using that account see the same records.

Main flow

1. Sign in and set the motorcycle's tank and gauge values.
2. Record a fill-up. Amount and date/time are required; liters, odometer, gauge level, station, fuel grade, and location are optional.
3. Review spending and fuel estimates on Home and Charts.
4. Use Log to edit or archive a record. Use Map to compare saved prices by fuel grade and see where visits are concentrated.

The app supports adding and editing fill-ups, station management, light/dark appearance, and paged lists. Records are archived rather than physically deleted. Map prices come from saved fill-ups; they are not live fuel offers.

Questions the app helps answer

- How has my fuel spending changed over time?
- What is my recent fuel economy?
- Which saved station has the lower average price for my fuel grade?

CASE STUDY

2. Technical implementation and mobile capabilities

The interface uses Ionic 8 and Angular 20. Capacitor 7 provides native access to Geolocation and Haptics. A PHP API validates requests and stores shared records in MySQL. On the device, Ionic Storage keeps a local snapshot and a queue of unsynchronized changes in IndexedDB. The application uses GET, POST, PUT, and DELETE operations for reading, creating, updating, and archiving records.

Data flow at a glance

Capture	Save locally	Synchronize	Shared data
The fill-up form creates a change	IndexedDB keeps the snapshot and retry queue	The app sends queued changes when online	The PHP API validates and writes them to MySQL

Location and feedback

The app requests location only after the user chooses to attach a current location to a fill-up or center the map. It does not track a person in the background. Permission denial, timeout, or unavailable location produces a message; the user can still save without a location. Haptic success feedback is optional and does not interrupt saving if unavailable. These flows are implemented in `src/app/services/geo.ts` [2, 5, 6].

```
if (Capacitor.isNativePlatform()) {
  await Geolocation.requestPermissions({ permissions: ['location'] });
}
const position = await Geolocation.getCurrentPosition({
  enableHighAccuracy: true,
  timeout: 10000,
  maximumAge: 30000,
});
return { latitude: position.coords.latitude, longitude: position.coords.longitude };
```

Local-first data operation

A fill-up is committed to local storage before the app attempts synchronization. This keeps a saved entry available while offline. When a connection returns, queued operations are sent to the API and the app refreshes its shared data. Operation IDs make safe retries possible when a response is lost. The server uses an authenticated session and CSRF checks for mutations; the API and database maintain a single shared dataset. This flow is implemented in `src/app/services/tracker-store.ts` [2].

```
await this.storage.set('snapshot', updated);
this.state.set(updated);
this.channel?.postMessage('updated');
// After the queued local change is committed:
void this.sync();
```

The database separates bike settings, station choices, fuel grades, fill-ups, and mutation receipts. Archived fill-ups retain their history. Local map tiles are not stored, so the map needs an internet connection.

CASE STUDY

3. Decision support and data calculations

Home and Charts summarize recorded purchases with price per liter, fuel economy, distance, monthly spending, and an estimated cost per kilometer. Fuel economy uses measured full-tank intervals when available, with a labeled gauge-based estimate as a fallback. Missing measurements are shown as unavailable instead of being treated as real zero values.

The cost-per-kilometer estimate follows the reference app's calculation: use the closing purchase's price per liter and divide by the measured efficiency for that interval. It is an estimate, not the actual cost of every kilometer traveled.

```
segment.to.amount / segment.to.liters / segment.efficiency;
```

This calculation is in `src/app/models/math.ts` [2]. The fill-up form records gauge level after refueling, and the app uses the configured low-fuel anchor and gauge segments when estimating fuel use.

Map comparison

The Map screen uses Leaflet with OpenStreetMap Standard tiles. A heat layer summarizes the frequency of recorded visits, while station cards compare historical average prices for the selected grade. The comparison groups nearby records for a station branch and distinguishes the latest usable price from the last visit. These are historical records, not current market prices [2, 7, 8].

Known limitations

- The class instance has one shared login and dataset, so changes are not attributed to individual classmates.
- Map tiles require internet access.
- Seed data supports manual exploration; some station, location, grade, time, liters, and odometer values are synthetic.
- Android is the intended target, but an installable APK and native device behavior have not been verified.

CASE STUDY

4. Testing and evaluation

Automated checks were run on November 7, 2026. Component tests use Angular's experimental test builder with Node and jsdom; map and canvas rendering are isolated. PHP integration checks use a disposable database and do not modify the shared `gas_tracker` database.

Feature or task	Expected result	Actual result	Status
Fuel calculations and offline queue	Calculations and local changes handle the covered cases	46 logic/offline tests passed	PASS
App components	Components handle the covered form, navigation, chart, settings, log, and map behavior	70 tests across 17 spec files passed in jsdom	PASS
PHP unit and migration behavior	Validation, migration, archival, and restore cases pass	28 tests and 393 assertions passed	PASS
Authenticated API behavior	API validates access and mutations against an isolated database	20 HTTP integration checks passed	PASS
Angular type check	App types compile without errors	`pnpm typecheck` passed	PASS
Main user flow in a browser	Sign in, add a fill-up, review screens, edit, and archive a record	No manual browser walkthrough was performed; visual layout is unverified	FUTURE IMPROVEMENT
Android install and native features	Install APK and verify location permission and haptics on a device	APK/device verification was not performed	FUTURE IMPROVEMENT

The automated results show that the tested logic, components, and API cases pass. They do not prove that the complete app flow works visually in a browser or on an Android device. The immediate next evaluation should build an APK and have a user complete the main flow on a device, including accepting and denying location permission.

If the app is extended beyond a class-shared dataset, add individual accounts and a change history so users can identify and recover each other's edits. This would address the current shared-account limitation.

AI usage disclosure

OpenAI Codex assisted with project scaffolding, implementation, debugging, automated tests, code review, and this report. The automated checks listed above were run. No manual browser walkthrough or Android device verification is claimed. Group members should review the selected code and be prepared to explain the app's behavior and limitations.

CASE STUDY

References

1. Mobile Application Technology - CS, Final Case Study brief supplied for the course.
2. Gas Tracker Mobile source repository: https://github.com/cedricangulo/gas-tracker-mobile. Code areas discussed: `src/app/services/geo.ts`, `src/app/services/tracker-store.ts`, and `src/app/models/math.ts`.
3. Ionic Framework documentation: https://ionicframework.com/docs.
4. Angular v20 component guide: https://v20.angular.dev/guide/components.
5. Capacitor Geolocation API, v7: https://capacitorjs.com/docs/v7/apis/geolocation.
6. Capacitor Haptics API, v7: https://capacitorjs.com/docs/v7/apis/haptics.
7. Leaflet API reference, v1.9.4: https://leafletjs.com/reference-1.9.4.html.
8. OpenStreetMap Foundation, Tile Usage Policy: https://operations.osmfoundation.org/policies/tiles/.

References accessed November 7, 2026.